

Doc. no.:	P2714R0
Date:	2022-1-12
Audience:	LEWG
Reply-to:	Zhihao Yuan <zy@miator.net> Tomasz Kamiński <tomaszkam@gmail.com>

Bind front and back to NTTP callables

- Bind front and back to NTTP callables
 - 1. Abstract
 - 2. Motivation
 - 3. Discussion
 - 3.1 Why not use a lambda?
 - 3.2 Should `std::bind` take an NTTP?
 - 3.3 Should `std::not_fn` take an NTTP?
 - 4. Design Decisions
 - 4.1 Extend the overload sets
 - 4.2 Reject targets of null pointers at compile-time
 - 5. Wording
 - 5.1 Feature test macro
 - 6. Implementation Experience
 - 7. References

1. Abstract

Users of `std::bind_front` and `std::bind_back` pay what they don't use when binding arguments to functions, member functions, and sometimes customization point objects. The paper proposes to allow passing callable objects as arguments for non-type template parameters in `bind_front` and `bind_back` to guarantee zero cost in those use cases. The paper also applies the same technique to `std::not_fn`.

2. Motivation

The need for partial function application^[1] is ubiquitous. Consider the following example modified from Qt 5:

C++20	<pre>connect(sender, &Sender::valueChanged, std::bind_front(&Receiver::updateValue, receiver, "Value"));</pre>
P2714	<pre>connect(sender, &Sender::valueChanged, std::bind_front<&Receiver::updateValue>(receiver, "Value"));</pre>

This paper proposes new overloads to `bind_front` and `bind_back` that move the binding target to template parameters for the following needs.

Reduce the cost of binding arguments to functions and member functions

Typically, binding arguments to a function using C++20 `std::bind_front` requires storing a function pointer along with the arguments, even though the language knows precisely which function to call without a need to dereference the pointer.

```
auto fn = std::bind_front(update_cb, receiver);  
static_assert(sizeof(fn) > sizeof(receiver));  
  
auto meth = std::bind_front(&Receiver::updateValue, receiver);  
static_assert(sizeof(meth) > sizeof(receiver));
```

Similarly, binding arguments to a member function often requires more storage and easily exceeds type-erased callback (such as `std::function`)'s SBO buffer capacity. But again, the compiler knows this call pattern at compile-time.

The call wrapper objects created by `bind_front` (and `bind_back`) are of unspecified types that are not meant to be assignable. In other words, the design consideration to support replacing the value of the object “of the same type” at run-time doesn’t exist. We, as users, don’t want to pay performance and space costs for something we don’t want to use and can’t use.

Simplify implementation and debug codegen

Captureless lambda gain traction as first-class functions that are safe to use and come with better defaults, which fall into the category of callables of empty classes. Ordinary users would expect binding arguments to empty objects to be indifferent from adding data

members to an empty class.

```
auto captureless = [](FILE *, void *, size_t) { return 0; };
static_assert(sizeof(std::bind_front(captureless, stdin)) == sizeof(stdin));
```

But what cost implementations to give this guarantee in `bind_front` ? Here is a breakdown:

Impl.	Layout
libc++	<pre>tuple<Target, BoundArgs...> state_entities;</pre>
MS STL	<pre>_Compressed_pair<Target, tuple<BoundArgs...>> state_entities;</pre>
libstdc++	<pre>[[no_unique_address]] Target target; tuple<BoundArgs...> bound_args;</pre>
P2714 †	<pre>[... bound_args(args)] // lambda captures</pre>

†A sample implementation can be found at the end of the article.

By not storing the call targets, this guarantee is by construction rather than a choice of QoI. Suppose an implementation takes a further step to lower `std::forward`, `std::forward_like`, and `std::invoke` as if they were intrinsics.^[2] There will be no intermediate layers to bloat debug information when accessing these binders' state entities.

Improve the readability of expressions that use `bind_front` and `bind_back`

The `invoke`-like call expressions such as `invoke(func, a, b, ...)` are becoming conventional in C++. However, unless you're a Lisp fan, you may still find that `func(a, b, ...)` is the "right" way and `bind_front<func>(a, b)` to be more natural. The latter form encodes necessary information for a casual reader to distinguish between the target and the bound arguments.

Such a visual distinction matters equally well to `bind_back<func>(a, b)`, as `bind_back(func, a, b)` has an implied "insertion point" for the actual arguments to the call wrapper.

3. Discussion

3.1 Why not use a lambda?

The question is probably not asking, “why not use a lambda in C++23,” since today’s lambda does not propagate `noexcept` and constraints, nor forwards captures and arguments easily. Assume that we will have a “perfect lambda” in C++26, where applying ‘`↵`’ in front of a parameter forwards the argument and laying ‘`↵`’ in front of a capture forwards the member using closure’s value category. `&...` is a parameter index pack that expands to `&1` , `&2` , ... So this will allow you to write

```
[=][do_work(↵bnd1, ↵bnd2, ↵&...)]
```

instead of

```
bind_front<do_work>(bnd1, bnd2)
```

Will I choose the “perfect lambda” instead? Maybe not. Not because the “perfect lambda” is not terse enough. The difference is every symbol in the `bind_front` expression is about **why** the call wrapper is needed here in the code, while every token in the “perfect lambda” is to explain **how** this is done. Partial application is an established idea; people will keep picking up `functools.partial` (<https://docs.python.org/3/library/functools.html#functools.partial>), although Python’s lambda is sufficiently clean and complete.

3.2 Should `std::bind` take an NTTP?

Compared to the later added `std::bind_front` and `std::bind_back` , `std::bind` is sub-optimal, and I do not want to encourage its uses. It is too sparse to find a use of `std::bind` that actually bound something but cannot be replaced by `bind_front` and `bind_back` . The unique value of `std::bind` comes from its capability of reordering arguments. Being said that, it is likely to be a lower-hanging fruit if we want to introduce a terse lambda that uses parameter indices.^[3]

3.3 Should `std::not_fn` take an NTTP?

A callback may want to negate its boolean result before being type-erased, and introducing `not_fn<f>()` seems to be an intuitive answer. But is that the only answer?

It's not hard to implement `not_fn` in a way such that the value of `not_fn(f)` is of a structural type if `f` is of a structural type. Therefore, if you demand a combination of partial application and negation whose call pattern is known at compile-time, you can use `bind_front<not_fn(f)>(a, b)`.

This can turn into a hostile workaround if I ask users to write `bind_front<not_fn(f)>()` when only negation is wanted; `not_fn<f>()` expresses the original intention directly.

Partial	C++20	P2714
Y	<code>bind_front(not_fn(f), a, b)</code>	<code>bind_front<not_fn<f>()>(a, b)</code>
N	<code>not_fn(f)</code>	<code>not_fn<f>()</code>

It also raises the question of whether we need an `fn<f>()` that encapsulates the `f`'s call pattern without negation. It can be deemed an NTPP version of `std::mem_fn`, except for not being limited to member pointers.

The paper proposes `not_fn<f>()`. Whether to require *perfect forward call wrappers* to propagate the structural property when all of their state entities are of structural types deserves a paper on its own, especially when a core issue (<https://github.com/cplusplus/CWG/issues/15>) is about to get involved.

4. Design Decisions

4.1 Extend the overload sets

Doing so allows the new APIs to reuse the existing names, `bind_front` and `bind_back`, enabling the users who are already familiar with these names to make a small change and see benefits.

As a design alternative, it was suggested that adding `operator()` to `std::nontype`^[4] can give a similar outcome:

alt.	<pre>connect(sender, &Sender::valueChanged, std::bind_front(std::nontype<&Receiver::updateValue>, receiver, "Value"));</pre>
P2714	<pre>connect(sender, &Sender::valueChanged, std::bind_front<&Receiver::updateValue>(receiver, "Value"));</pre>

The soundness behind the suggestion was that all vendors[‡] had implemented EBO in `bind_front`; that is to say, the specializations of `nontype_t` could be optimized without effort. However, as shown at the beginning of the article, “no effort” means a lot of effort – to generate code and debug information. Though an implementation can create overloads to treat `nontype_t<f>` differently, the users get the same thing, only with a convoluted spelling.

The design decision extends to `not_fn` as well. `std::not_fn` is a function template, meaning the same name cannot be redeclared as a variable. Unsurprisingly, keeping users’ familiarity with the existing name outweighs the benefit of saving a pair of parentheses.

[‡]Except for `libstdc++` until GCC 13. See [Compiler Explorer aee1P35b4](https://compiler-explorers.com/posts/aee1P35b4/) (<https://godbolt.org/z/aee1P35b4>).

4.2 Reject targets of null pointers at compile-time

You won’t get a null pointer to function or a null pointer to member via a function-to-pointer conversion or an `&` operator in this language. Still, the targets of these types may be computed at compile-time:

```
// error: static_assert failed: 'f != nullptr'
std::bind_front<tbl.get("nonexistent-callback")>(a);
```

The new overloads have such information ahead of time and can easily diagnose it. `function_ref`’s constructor that initializes from `nontype<f>` applies the identical practice.

5. Wording

The wording is relative to N4917 (<https://wg21.link/N4917>).

Append the following to `[func.bind.partial]` (<http://www.eelis.net/c++draft/func.bind.partial>), function templates `bind_front` and `bind_back`:

```
template<auto f, class... Args>
    constexpr unspecified bind_front(Args&&... args);
template<auto f, class... Args>
    constexpr unspecified bind_back(Args&&... args);
```

Within this subclause:

- *F* is the type of *f* ,
- *g* is a value of the result of a *bind_front* or *bind_back* invocation,
- *BoundArgs* is a pack that denotes `decltype(auto(args))...` ,
- *bound_args* is a pack of bound argument entities of *g* ([*func.def*] (<http://www.eelis.net/c++draft/func.def>)) of types *BoundArgs*... , direct-non-list-initialized with `std::forward<Args>(args)...` , respectively, and
- *call_args* is an argument pack used in a function call expression ([*expr.call*] (<http://www.eelis.net/c++draft/expr.call>)) of *g* .

Mandates:

- `(is_constructible_v<BoundArgs, Args> && ...)` is true , and
- `(is_move_constructible_v<BoundArgs> && ...)` is true , and
- if `is_pointer_v<F> || is_member_pointer_v<F>` is true , then `f != nullptr` is true .

Preconditions: For each *T_i* in *BoundArgs* , if *T_i* is an object type, *T_i* meets the *Cpp17MoveConstructible* requirements.

Returns: A perfect forwarding call wrapper ([*func.require*]

(<http://www.eelis.net/c++draft/func.require#term.perfect.forwarding.call.wrapper>)) *g* that does not have target object, and has call pattern:

- `invoke(f, bound_args..., call_args...)` for a *bind_front* invocation, or
- `invoke(f, call_args..., bound_args...)` for a *bind_back* invocation.

Throws: Any exception thrown by the initialization of *bound_args* .

Append the following to [*func.not.fn*] (<http://www.eelis.net/c++draft/func.not.fn>), function template *not_fn* :

```
template<auto f> constexpr unspecified not_fn() noexcept;
```

In the text that follows:

- `F` is the type of `f`,
- `g` is a value of the result of a `not_fn` invocation,
- `call_args` is an argument pack used in a function call expression (`[expr.call]` (<http://www.eelis.net/c++draft/expr.call>)) of `g`.

Mandates: If `is_pointer_v<F> || is_member_pointer_v<F>` is true, then `f != nullptr` is true.

Returns: A perfect forwarding call wrapper (`[func.require]` (<http://www.eelis.net/c++draft/func.require#term.perfect.forwarding.call.wrapper>)) `g` that does not have state entities, and has call pattern `!invoke(f, call_args...)`.

Add the signatures to `[functional.syn]` (<https://eel.is/c++draft/functional.syn>), header `<functional>` synopsis:

[...]

```
// [func.not.fn], function template not_fn
template<class F> constexpr unspecified not_fn(F&& f);           // freestanding
template<auto f> constexpr unspecified not_fn() noexcept;     // freestanding

// [func.bind.partial], function templates bind_front and bind_back
template<class F, class... Args>
    constexpr unspecified bind_front(F&&, Args&&...);         // freestanding
template<class F, class... Args>
    constexpr unspecified bind_back(F&&, Args&&...);          // freestanding

template<auto f, class... Args>
    constexpr unspecified bind_front(Args&&...);              // freestanding
template<auto f, class... Args>
    constexpr unspecified bind_back(Args&&...);               // freestanding
```

[...]

5.1 Feature test macro

Update values in `[version.syn]` (<https://eel.is/c++draft/version.syn>), header `<version>` synopsis:


```

#define __cpp_lib_bind_back          202202L20XXXXL // also in <functional>
#define __cpp_lib_bind_front        201907L20XXXXL // also in <functional>
[...]
```

```

#define __cpp_lib_not_fn             201603L20XXXXL // also in <functional>
```

6. Implementation Experience

The snippet below is a full implementation of the proposed `bind_front` overload. You can play with this and the rest of the proposal in [aGbTe8frj](https://godbolt.org/z/aGbTe8frj) Compiler Explorer (<https://godbolt.org/z/aGbTe8frj>).

```

template<class T, class U>
struct __copy_const : conditional<is_const_v<T>, U const, U> {};

template<class T, class U,
        class X = __copy_const<remove_reference_t<T>, U>::type>
struct __copy_value_category
    : conditional<is_lvalue_reference_v<T&&>, X&, X&&> {};

template<class T, class U>
struct type_forward_like : __copy_value_category<T, remove_reference_t<U>> {};

template<class T, class U>
using type_forward_like_t = type_forward_like<T, U>::type;

template<auto f, class... Args>
constexpr auto bind_front(Args&&... args) {
    using F = decltype(f);
    if constexpr (is_pointer_v<F> or is_member_pointer_v<F>)
        static_assert(f != nullptr);
    return
        [... bound_args(std::forward<Args>(args))]<class Self, class... T>(
            this Self&&,
            T&&... call_args)
    noexcept(is_nothrow_invocable_v<
        F, type_forward_like_t<Self, decay_t<Args>>..., T...>)
        -> invoke_result_t<F, type_forward_like_t<Self, decay_t<Args>>...,
            T...> {
        return std::invoke(f, std::forward_like<Self>(bound_args)...,
            std::forward<T>(call_args)...);
    };
}
```

7. References

1. Kamiński, Tomasz. P0356R5 *Simplified partial function application*. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0356r5.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0356r5.html>) ↩
2. DaCamara, Cameron. *Improving the State of Debug Performance in C++*. <https://devblogs.microsoft.com/cppblog/improving-the-state-of-debug-performance-in-c/> (<https://devblogs.microsoft.com/cppblog/improving-the-state-of-debug-performance-in-c/>) ↩
3. Vector of Bool. *A Macro-Based Terse Lambda Expression*. <https://vector-of-bool.github.io/2021/04/20/terse-lambda-macro.html> (<https://vector-of-bool.github.io/2021/04/20/terse-lambda-macro.html>) ↩
4. Romeo, et al. P0792R13 *function_ref: a type-erased callable reference*. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0792r13.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0792r13.html>) ↩